```
import requests
from bs4 import BeautifulSoup
import json
import os
import time
from datetime import datetime, timezone, timedelta

RANKING_URL = 'https://www.dlsite.com/girls-touch/ranking/day'
HISTORY_FILE = 'ranking_history.json'
RANKING_FILE = 'ranking_data.json'
TAG_FILE = 'tag_ranking.json'
INDEX_FILE = 'index.html'
MAX_HISTORY_DAYS = 90

JST = timezone(timedelta(hours=9))
now = datetime.now(JST)
date_str = now.strftime('%Y-%m-%d %H:%M')
date_key = now.strftime('%Y-%m-%d')

HEADERS = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
    'Accept-Language': 'ja,en;q=0.9',
}

def scrape_ranking():
    res = requests.get(RANKING_URL, headers=HEADERS, timeout=30)
    res.encoding = 'utf-8'
    soup = BeautifulSoup(res.text, 'lxml')
    rows = soup.select('tr.ranking_list_item')
    items = []
    for row in rows:
        try:
            rank_el = row.select_one('td.ranking_num')
            title_el = row.select_one('dt.work_name a')
            if not rank_el or not title_el:
                continue
            rank = int(rank_el.text.strip())
            title = title_el.text.strip()
            url = title_el.get('href', '')
            if url and url.startswith('//'):
                url = 'https:' + url
            img_el = row.select_one('img')
            img = ''
            if img_el:
```

```
                img = img_el.get('src') or img_el.get('data-src') or ''
                if img.startswith('//'):
                    img = 'https:' + img
            items.append({'rank': rank, 'title': title, 'url': url, 'img': img,
                          'circle': '', 'author': '', 'release_date': '', 'dl_cou
        except Exception as e:
            print('Row error: ' + str(e))
            continue
    return items

def scrape_detail(url):
    try:
        time.sleep(0.8)
        res = requests.get(url, headers=HEADERS, timeout=20)
        res.encoding = 'utf-8'
        soup = BeautifulSoup(res.text, 'lxml')
        circle = ''
        author = ''
        release_date = ''
        dl_count = 0
        price = 0
        tags = []
        circle_el = soup.select_one('span.maker_name a') or soup.select_one('a.ma
        if circle_el:
            circle = circle_el.text.strip()
        for tr in soup.select('table#work_outline tr'):
            th = tr.select_one('th')
            td = tr.select_one('td')
            if not th or not td:
                continue
            label = th.text.strip()
            if '\u4f5c\u8005' in label or '\u8457\u8005' in label:
                author = td.text.strip()
            elif '\u8ca9\u58f2\u65e5' in label or '\u66f4\u65b0\u65e5' in label:
                release_date = td.text.strip()
            elif '\u30c0\u30a6\u30f3\u30ed\u30fc\u30c9' in label or 'DL\u6570' in
                try:
                    dl_count = int(td.text.strip().replace(',', '').replace('\u4e
                except:
                    pass
        price_el = soup.select_one('span.price') or soup.select_one('.work_price'
        if price_el:
            try:
                price = int(price_el.text.strip().replace(',', '').replace('\u518
            except:
                pass
        for a in soup.select('div.main_genre a, a.genre_tag, div.work_genre a'):
```

```
            t = a.text.strip()
            if t:
                tags.append(t)
        return {'circle': circle, 'author': author, 'release_date': release_date,
                'dl_count': dl_count, 'price': price, 'tags': tags}
    except Exception as e:
        print('Detail error ' + url + ': ' + str(e))
        return {}

def build_tag_ranking(items):
    tag_count = {}
    for item in items:
        for tag in item.get('tags', []):
            tag_count[tag] = tag_count.get(tag, 0) + 1
    sorted_tags = sorted(tag_count.items(), key=lambda x: x[1], reverse=True)
    return {'tags': [{'tag': t, 'count': c} for t, c in sorted_tags[:30]]}

def load_history():
    if os.path.exists(HISTORY_FILE):
        with open(HISTORY_FILE, 'r', encoding='utf-8') as f:
            return json.load(f)
    return []

def save_history(history, new_entry):
    history = [d for d in history if d.get('date_key') != date_key]
    history.append(new_entry)
    history = sorted(history, key=lambda x: x['date_key'])[-MAX_HISTORY_DAYS:]
    with open(HISTORY_FILE, 'w', encoding='utf-8') as f:
        json.dump(history, f, ensure_ascii=False, indent=2)
    return history

def build_index_html(ranking_data, tag_data, history_data):
    r_json = json.dumps(ranking_data, ensure_ascii=False)
    t_json = json.dumps(tag_data, ensure_ascii=False)
    h_json = json.dumps(history_data, ensure_ascii=False)
    html = '''<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>\u304c\u308b\u307e\u306b\u30e9\u30f3\u30ad\u30f3\u30b0</title>
<link href="https://fonts.googleapis.com/css2?family=Noto+Sans+JP:wght@400;700;90
<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.4.1/chart.umd.min.
<style>
:root{--pink:#ff4da6;--pink-light:#ff80c8;--purple:#b44dff;--bg:#0d0010;--bg2:#13
*{margin:0;padding:0;box-sizing:border-box;}
body{background:var(--bg);color:var(--text);font-family:'Noto Sans JP',sans-serif
```

```
body::before{content:'';position:fixed;inset:0;background:linear-gradient(rgba(18
header{position:sticky;top:0;z-index:100;background:rgba(13,0,16,0.92);backdrop-f
.header-inner{display:flex;align-items:center;justify-content:space-between;max-w
.logo{font-family:'Kaisei Decol',serif;font-size:20px;background:linear-gradient(
.last-updated{font-size:10px;color:var(--text-muted);}
.tabs{display:flex;gap:8px;padding:14px 16px 0;max-width:600px;margin:0 auto;over
.tabs::-webkit-scrollbar{display:none;}
.tab{flex-shrink:0;padding:7px 16px;border-radius:20px;font-size:13px;font-weight
.tab.active{background:linear-gradient(135deg,var(--pink),var(--purple));border-c
main{max-width:600px;margin:0 auto;padding:14px 12px 80px;position:relative;z-ind
@keyframes fadeUp{from{opacity:0;transform:translateY(12px);}to{opacity:1;transfo
@keyframes neonPulse{0%,100%{box-shadow:0 0 8px rgba(255,215,0,0.3);}50%{box-shad
.rank-card{background:var(--card-bg);border:1px solid var(--card-border);border-r
.rank-card:active{transform:scale(0.98);}
.rank-card.mega-hit{border-color:rgba(255,215,0,0.6);animation:fadeUp 0.3s ease b
.thumb-wrap{width:90px;min-width:90px;height:120px;overflow:hidden;position:relat
.thumb-wrap img{width:100%;height:100%;object-fit:cover;display:block;}
.thumb-placeholder{width:100%;height:100%;display:flex;align-items:center;justify
.rank-badge{position:absolute;top:6px;left:6px;width:26px;height:26px;border-radi
.rank-badge.r1{background:linear-gradient(135deg,#ffd700,#ffaa00);color:#000;}
.rank-badge.r2{background:linear-gradient(135deg,#c0c0c0,#888);color:#000;}
.rank-badge.r3{background:linear-gradient(135deg,#cd7f32,#8b4513);color:#fff;}
.rank-badge.other{background:rgba(0,0,0,0.7);color:var(--text-muted);border:1px s
.card-info{flex:1;padding:10px 12px;display:flex;flex-direction:column;gap:5px;mi
.card-title{font-size:13px;font-weight:700;line-height:1.4;display:-webkit-box;-w
.card-circle{font-size:11px;color:var(--pink-light);white-space:nowrap;overflow:h
.card-meta{display:flex;gap:10px;flex-wrap:wrap;}
.meta-item{font-size:11px;color:var(--text-muted);}
.meta-item.dl{color:var(--pink-light);font-weight:700;}
.meta-item.price{color:#aaffcc;}
.card-tags{display:flex;gap:4px;flex-wrap:wrap;margin-top:2px;}
.tag{font-size:10px;padding:2px 7px;border-radius:10px;background:rgba(180,77,255
.section-title{font-family:'Kaisei Decol',serif;font-size:16px;color:var(--pink);
.section-title::after{content:'';flex:1;height:1px;background:linear-gradient(90d
.tag-rank-list{display:flex;flex-direction:column;gap:8px;}
.tag-rank-item{display:flex;align-items:center;gap:10px;background:var(--card-bg)
.tag-rank-num{font-size:18px;font-weight:900;min-width:28px;color:var(--purple);}
.tag-rank-num.t1{color:#ffd700;}
.tag-rank-num.t2{color:#c0c0c0;}
.tag-rank-num.t3{color:#cd7f32;}
.tag-rank-name{flex:1;font-size:14px;font-weight:700;}
.tag-rank-bar-wrap{width:80px;}
.tag-rank-bar{height:6px;border-radius:3px;background:linear-gradient(90deg,var(-
.tag-rank-count{font-size:11px;color:var(--text-muted);margin-top:3px;text-align:
.chart-card{background:var(--card-bg);border:1px solid var(--card-border);border-
.chart-card canvas{max-height:220px;}
.chart-note{font-size:10px;color:var(--text-muted);text-align:center;margin-top:8
```

```
footer{position:fixed;bottom:0;left:0;right:0;background:rgba(13,0,16,0.95);borde
.heart-particle{position:fixed;pointer-events:none;z-index:9999;animation:heartFl
@keyframes heartFloat{0%{opacity:1;transform:translateY(0) scale(1);}100%{opacity
</style>
</head>
<body>
<header>
  <div class="header-inner">
    <div class="logo">&#x1F49C; \u304c\u308b\u307e\u306b\u30e9\u30f3\u30ad\u30f3\
    <div class="last-updated" id="lastUpdated"></div>
  </div>
</header>
<div class="tabs">
  <div class="tab active" onclick="switchTab('ranking')">\u{1F3C6} \u30e9\u30f3\u
  <div class="tab" onclick="switchTab('tags')">\u{1F3F7} \u30bf\u30b0\u4eba\u6c17
  <div class="tab" onclick="switchTab('graph')">\u{1F4C8} \u63a8\u79fb\u30b0\u30e
</div>
<main>
  <div id="rankingSection"></div>
  <div id="tagsSection" style="display:none"></div>
  <div id="graphSection" style="display:none"></div>
</main>
<footer>\u30c7\u30fc\u30bf\u306f\u6bce\u65e5\u6df1\u591c0\u6642\u9803\u306b\u81ea
<script>
var RANKING_DATA = ''' + r_json + ''';
var TAG_DATA = ''' + t_json + ''';
var HISTORY_DATA = ''' + h_json + ''';
var chartInstances = {};

function esc(s) {
  var d = document.createElement('div');
  d.textContent = String(s || '');
  return d.innerHTML;
}

function switchTab(tab) {
  var names = ['ranking','tags','graph'];
  document.querySelectorAll('.tab').forEach(function(el,i){ el.classList.toggle('
  names.forEach(function(n){ document.getElementById(n+'Section').style.display =
}

function spawnHearts(x,y) {
  var hearts = ['\u2665','\u2764','\u2728','\u2605','\u25cf'];
  for(var i=0;i<5;i++){
    (function(idx){
      setTimeout(function(){
        var el = document.createElement('div');
```

```
        el.className = 'heart-particle';
        el.textContent = hearts[Math.floor(Math.random()*hearts.length)];
        el.style.left = (x+(Math.random()-0.5)*60)+'px';
        el.style.top = (y+(Math.random()-0.5)*20)+'px';
        el.style.fontSize = (14+Math.random()*14)+'px';
        el.style.color = ['#ff4da6','#b44dff','#ff80c8','#ffd700'][Math.floor(Mat
        document.body.appendChild(el);
        setTimeout(function(){ el.remove(); }, 1300);
      }, idx*80);
    })(i);
  }
}
document.addEventListener('click', function(e){ if(e.target.closest('.rank-card')

function renderRanking(data) {
  document.getElementById('lastUpdated').textContent = data.date ? data.date+' \u
  var items = data.items || [];
  var html = '';
  items.forEach(function(item,idx){
    var isMega = (item.dl_count||0) >= 10000;
    var tags = (item.tags||[]).slice(0,4);
    var dlStr = item.dl_count ? item.dl_count.toLocaleString()+'DL' : '';
    var priceStr = item.price ? item.price.toLocaleString()+'\u5186' : '';
    var circle = item.circle || item.author || '';
    var badge = item.rank<=3 ? 'r'+item.rank : 'other';
    var imgHtml = item.img ? '<img src="'+item.img+'" alt="" loading="lazy" onerr
    html += '<a class="rank-card'+(isMega?' mega-hit':'')+'" href="'+(item.url||'
    html += '<div class="thumb-wrap">'+imgHtml+'<div class="rank-badge '+badge+'"
    html += '<div class="card-info"><div class="card-title">'+esc(item.title)+'</
    if(circle) html += '<div class="card-circle">'+esc(circle)+'</div>';
    html += '<div class="card-meta">';
    if(dlStr) html += '<span class="meta-item dl">'+esc(dlStr)+(isMega?' <span st
    if(priceStr) html += '<span class="meta-item price">'+priceStr+'</span>';
    if(item.release_date) html += '<span class="meta-item">'+esc(item.release_dat
    html += '</div>';
    if(tags.length){ html += '<div class="card-tags">'; tags.forEach(function(t){
    html += '</div></a>';
  });
  document.getElementById('rankingSection').innerHTML = html;
}

function renderTags(data) {
  var tags = data.tags || [];
  var max = tags.length ? tags[0].count : 1;
  var html = '<div class="section-title">\u{1F3F7} \u4eca\u65e5\u306e\u4eba\u6c17
  tags.forEach(function(t,i){
    var cls = i<3 ? 't'+(i+1) : '';
```

```
    html += '<div class="tag-rank-item" style="animation-delay:'+i*0.05+'s">';
    html += '<div class="tag-rank-num '+cls+'">'+(i+1)+'</div>';
    html += '<div class="tag-rank-name">'+esc(t.tag)+'</div>';
    html += '<div class="tag-rank-bar-wrap"><div class="tag-rank-bar" style="widt
    html += '<div class="tag-rank-count">'+t.count+'\u4f5c\u54c1</div></div></div
  });
  html += '</div>';
  document.getElementById('tagsSection').innerHTML = html;
}

function renderGraph(history) {
  if(!history||history.length===0){
    document.getElementById('graphSection').innerHTML='<div style="text-align:cen
  }
  var tagByDate={};
  history.forEach(function(day){
    tagByDate[day.date_key]={};
    (day.items||[]).forEach(function(item){
      (item.tags||[]).forEach(function(tag){ tagByDate[day.date_key][tag]=(tagByD
    });
  });
  var dates=Object.keys(tagByDate).sort();
  var totalTagCount={};
  dates.forEach(function(d){ Object.entries(tagByDate[d]).forEach(function(e){ to
  var topTags=Object.entries(totalTagCount).sort(function(a,b){return b[1]-a[1];}
  var colors=['#ff4da6','#b44dff','#ff80c8','#7a1fa2','#ff9de2','#d084ff','#ffaad
  var datasets=topTags.map(function(tag,i){
    return {label:tag,data:dates.map(function(d){return tagByDate[d][tag]||0;}),b
  });
  if(chartInstances.tagTrend) chartInstances.tagTrend.destroy();
  document.getElementById('graphSection').innerHTML=
    '<div class="section-title">\u{1F4C8} \u30bf\u30b0\u4eba\u6c17\u306e\u63a8\u7
    '<div class="chart-card"><canvas id="chartTagTrend"></canvas>'+
    '<div class="chart-note">\u904e\u53bb'+dates.length+'\u65e5\u9593\u306e\u63a8
  chartInstances.tagTrend=new Chart(document.getElementById('chartTagTrend').getC
    type:'line',data:{labels:dates.map(function(d){return d.slice(5);}),datasets:
    options:{responsive:true,plugins:{legend:{position:'bottom',labels:{color:'#d
  });
}

renderRanking(RANKING_DATA);
renderTags(TAG_DATA);
renderGraph(HISTORY_DATA);
</script>
</body>
</html>'''
    return html
```

```
# --- main ---
print('スクレイピング開始...')
items = scrape_ranking()
print(str(len(items)) + '件取得')

for item in items:
    print('詳細取得: ' + str(item['rank']) + '位 ' + item['title'][:20])
    detail = scrape_detail(item['url'])
    item.update(detail)

ranking_data = {'date': date_str, 'date_key': date_key, 'items': items}
tag_data = build_tag_ranking(items)

with open(RANKING_FILE, 'w', encoding='utf-8') as f:
    json.dump(ranking_data, f, ensure_ascii=False, indent=2)

with open(TAG_FILE, 'w', encoding='utf-8') as f:
    json.dump(tag_data, f, ensure_ascii=False, indent=2)

history = load_history()
history = save_history(history, ranking_data)

html = build_index_html(ranking_data, tag_data, history)
with open(INDEX_FILE, 'w', encoding='utf-8') as f:
    f.write(html)

print('完了！index.html生成済み')
```